

INDEX

A

- aarch64, 140
- aarch64_16k, 140
- abusing accessibility services, 55
- ACK (Android Common Kernel), 124
- Activity Manager, 44
- ActivityManager, 111
- ADB (Android Debug Bridge), 15, 22
- AddressSanitizer (ASan), 11, 28, 95, 126
- address space layout randomization (ASLR), 106
- adler32, 83
- Advanced Protection mode, 47
- AFL++ fuzzing engine, 22, 25–26, 29–30, 78, 181
 - CMPLOG, 180
- AI automation, 175
- AIDL (Android Interface Definition Language), 53, 116
- AI models, fuzzing, 176–178. *See also*
 - auto-triage
 - automating the fuzzing process, 178–179
 - challenges, 179
 - automating structure-aware mutation, 182
 - gray-box fuzzing, 179–181
 - LLM-generated inputs, 181–182
 - white-box fuzzing, 179–181
- American Fuzzy Lop (AFL) engine, 29
- Android
 - architecture, layers of, 43–44
 - fragmentation, 51–53
 - hardware-based security, 47–51
 - permission model, 43, 53–55
 - security evolution, 45–47
 - SELinux, 56–60
- Android.bp* files, 19, 31–33
- Android build system, 21, 79
- Android Common Kernel (ACK), 124
- Android Debug Bridge (ADB), 15, 22
- Android Interface Definition Language (AIDL), 53, 116
- Android Keystore, 48–50
- AndroidManifest.xml* file, 53
- Android Open Source Project (AOSP), 15
- Android Package Kit (APK) file, 60
- Android runtime (ART), 44
- Android Security Bulletins, 70
- Android Vulnerability Rewards Program, 104
- AOSP (Android Open Source Project), 15
- API contract enforcement, 111–113
- API-to-service mapping, 67
- APK Signature Scheme v2, 60
- app_data_file type, 57
- application framework, 44
- app signing, 60
- ARM-based processors, 48
- ART (Android runtime), 44
- ASan (AddressSanitizer), 11, 28, 95, 126
- ASCII art, 177
- ASLR (address space layout randomization), 106
- assert statement, 98
- atof(), 74
- atoi(), 74
- atol(), 74
- atoll(), 74
- attack vector
 - local, 103
 - physical, 104
 - proximal, 103
 - remote, 103
- audioserver, 59
- auto-triage, 183
 - crash de-duplication, 183
 - remediation, 184
 - root cause analysis, 183

B

- .bashrc* file, 16, 22
- BeOS, 113
- binary instrumentation, 9
- Binder, 75, 109
 - kernel driver, 110–111
- binder2corpus, 121
- binder_parcel_fuzzer, 113–116, 119–120
- BiometricPrompt API, 51
- biometrics, 48, 51
- Bionic, 25
- black-box fuzzing, 7, 110–111. *See also*
 - gray-box fuzzing; white-box fuzzing
- blueprint files, 19
- Bluetooth, 103–104, 172
- bootloader, 103
- BpBinder, 111
- branch, 7
- bug bounty researcher, 100
- bugs
 - detection, 10–12
 - reproduction, 96–97
 - severity assessment, 100–106
 - triage, 93–106
- build.sh* shell script, 125
- build targets, 19
- bypassing permission checks, 56
- bytecode of Android apps, 44
- bzImage* file, 125

C

- camera server, 117–118
- cc_fuzz, 19–20, 78, 83, 91
- cc_library, 19–20, 78
- CERT C Coding Standard, 74
- cflags, 79
- checkSelfPermission(), 55
- Chinese Android ecosystem, 52
- clear_code(), 149
- client codebase, 111
- ClusterFuzz, 37–38
- cmpcov, 6
- CMPLOG, 180
- code
 - coverage instrumentation, 9
 - execution, 101
 - footprint, 48
 - under test, 7

- CodeMender, 184
- cognitive complexity, 73
- common-android-mainline*, 124
- common-modules*, 155
- compile-time instrumentation, 9, 29
- constrained process, 103
- ConsumeBool(), 116
- Content Providers, 44
- contributor health, 75
- copy_from_user(), 147, 149
- corpus, 29
 - binder2corpus, 121
 - collection, 89–90
 - minimization, 37
 - seed corpus, 90–91
- coverage-guided fuzzing, 7, 9, 118
- coverage reports, 38–42
- C++ vtables, 101
- crash analysis, 93. *See also* severity assessment
 - crash de-duplication, 99–100
 - initial validation, 94–99
- criticality score, 70–73
- cross-site script inclusion (XSSI)
 - attacks, 68
- cryptographic hashes, 6
- Cuttlefish
 - emulator, 16–18
 - kernel fuzzing with, 124–125
 - Syzkaller configuration, 128–130
 - testing with, 23–24
- cuttlefish-basic.json*, 157
- CyberArk Labs, 177–178
- cyclic redundancy checks (CRCs), 6
- cyclomaticity, 73

D

- DAC (discretionary access control), 56
- data-handling tools, 27, 86
- data parsing, 31–37
 - FuzzedDataProvider library, 34–35
 - libprotobuf-mutator library, 35–37
 - manual parsing, 32–34
- ddk_module, 140
- debugging
 - debug information, 94–95, 106
 - host environments, 168–169

- kernel logging, 159–161
- symbols, 39, 95
- DEFINE_PROTO_FUZZER macro, 36–37
- denial of service (DoS) attacks, 10, 102
- de-prioritization, 97
- device setup, 16–17
 - emulated devices, 18
 - physical devices, 17–18
- dictionary file, adding, 88–89
- digital rights management (DRM), 48
- disable_syscalls, 158–159
- disabling checks, 7
- discretionary access control (DAC), 56
- Docker, 127
- domains, SELinux, 56–57
- double-free vulnerabilities, 101
- dynamic instrumentation, 30

E

- empty fuzz harness, 78–80
- enable_syscalls, 158
- encryption, 45, 50
- error logging, disabling, 84–85
- example.pb.h*, 35
- example.proto*, 35–36
- Exploit.AndroidOS.Lotoor attack, 45
- export_header_lib_headers, 20
- exposure to third-party applications, 67
- Extensible Markup Language (XML), 5
- extensive physical access, 104

F

- fd_lockbox, 150–151
- fdp_fuzzer.cpp*, 34–35
- feedback-driven fuzzing. *See* coverage-guided fuzzing
- fillRandomParcel(), 115
- fingerprint readers, 138, 145
- flags for syz-extract, 154
- flags parameter, 151
- fmemopen(), 81
- _fops object, 146
- fork server, 29
- format_fuzzer, 19, 21
- fragmentation, 51–53
 - Chinese Android ecosystem, 52
 - Project Treble, 52–53
- Frida, 30

- full-disk encryption, 45
- function pointers, 5, 101
- FuzzedDataProvider library, 34–35, 178
- fuzz harnesses
 - creating, 31–33
 - definition, 7
 - empty, 78–80
 - running, 80
 - Soong integration, 19
- fuzzing. *See also* fuzzing binder services; fuzzing a library
 - AI models, 176–178
 - definition, 4
 - false positives, 98
 - large-scale systems, 109–122
 - life cycle, automating, 37–38
 - vs. other testing types, 12–13
 - overview, 1–2
 - process, 4–12
 - detecting bad behaviors, 10–12
 - evaluating progress, 7–10
 - generating random inputs, 4–7
 - running code under test, 7
- fuzzing a library
 - creating a fuzz harness, 78–80
 - disabling error logging, 84–85
 - fixing memory leaks, 86–87
 - implementing library code, 80–84
 - improving performance, 88–91
- fuzzing binder services
 - automating a service fuzzer, 115–117
 - fuzzing multiple services, 117–119
 - fuzzing parcel objects, 113–115
 - optimizing coverage, 119–120
 - scaling to multiple services, 121
 - testing the fuzzer framework, 122
- fuzzing engines, 6–7, 29–31. *See also* coverage-guided fuzzing
 - AFL++, 29–30
 - LibAFL, 30–31
 - libFuzzer, 29
 - Syzkaller, 30
- fuzzing environment
 - Android Debug Bridge (ADB), 22
 - Android source, downloading, 15–16
 - device setup
 - emulated devices, 18
 - physical devices, 17–18

- fuzzing environment (*continued*)
 - Soong build system, 19–22
 - testing
 - with AFL++, 25–26
 - with Cuttlefish, 23–24
 - on a host machine, 25
 - on a physical device, 24–25
 - fuzzService(), 115–116
 - fuzz_service_test, 122
 - FuzzyAI, 177

G

- generative fuzzing, 5
- genetic attack, 178
- getRandomBinder(), 115
- getRandomFds(), 115
- gets(), 74
- Ghidra, 8
- GingerBreak, 45
- GitHub, 127
- Gitiles REST API, 68
- Git repositories
 - Android source, 16–17
 - contributor metrics, 69
- global offset table (GOT), 45
- GNU Compiler Collection (GCC), 30
- Goldfish, 16–17
- grammar-based fuzzing, 5, 182
- granular app permission system, 45
- gray-box fuzzing, 7, 179–182. *See also*
 - black-box fuzzing; white-box fuzzing

H

- hallucination-based attack, 177–178
- hardware abstraction layer (HAL), 44, 53
- hardware-based security, 47–51
 - protected virtual machines, 51
 - secure elements, 50
 - trusted execution environments (TEEs), 47–50
- hardware-enforced walls, 51
- hardware interface definition language (HIDL), 53
- HarmonyOS, 52
- harness. *See* fuzz harnesses
- header_libs, 20
- Heartbleed bug, 4, 101
- HID (human interface device), 133–134

- hope-based programming, 112
- host-based fuzzing
 - advantages, 168–169
 - challenges, 169–170
 - enabling, 170
 - for all dependencies, 171
 - mocking platform-specific components, 172–173
 - in Linux environments, 173–174
 - overview, 167–168
- host machine, 25
- host_supported build flag, 170–171
- Huawei, 52
- human interface device (HID),
 - 133–134
- hwaddress, 22, 24
- HWASan, 28
- hybrid fuzzing, 181–182
- HyLLfuzz, 179, 181–182, 184
- hypervisor, 103

I

- important flags, 101
- infinite loops, 10
- info_ptr, 86
- information disclosure, 101–102
- initial crash validation
 - analyzing crash information, 94–95
 - confirming legitimate bugs, 98–99
 - reproducing crashes, 96–97
- initramfs.img, 125
- install-time permissions, 54
- insufficient de-duplication, 100
- integration testing, 12–13
- intellectual property (IP), 174
- inter-process communication (IPC),
 - 56, 110
- invariant-based fuzzing, 11
- ioctl syscall, 149
- _IOW() macros, 145

J

- jailbreaks, 50, 176
- Java archive file (JAR) signing, 60
- Java Native Interface (JNI), 67
- Java programming language libraries, 44
- JavaScript Object Notation (JSON), 35,
 - 71, 128–129

- j flag, 21
- JNINativeMethod registration, 67
- just-in-time (JIT) compiler, 9

K

- KASAN (Kernel Address Sanitizer), 159
- KCOV, 126–127
- kernel-based virtual machine (KVM), 18
- kernel fuzzing, 123–135
 - downloading and building kernel code, 124–126
 - fuzzing instrumentation, 126
 - Linux Kernel Library (LKL), 132–135
 - fuzz harness, 133
 - high-level workflow, 134
 - human interface device, 133–134
 - LKL-based fuzzer, 133
 - vs. Syzkaller, 135
 - preparation, 124
 - Syzkaller, 126–131
 - configuration, 128–131
 - vs. LKL, 135
 - overview, 128
 - sandboxed environment, fuzzing
 - inside, 131–132
 - setting up, 127–128
 - kernel-headers.tar.gz*, 154
 - KeyGenParametersSpec, 50
 - key management, 48
 - Keymaster HAL, 50
 - Keystore system, 50
 - KVM (kernel-based virtual machine), 18

L

- language runtime, 111
- large language models (LLMs), 175–184
 - bug triage
 - crash de-duplication, 183
 - remediation, 183–184
 - root cause analysis, 183–184
 - fuzzing, 176–177
 - automating, 178–179
 - jailbreaks, 175–177
 - LLM-as-judge method, 177
 - large-scale fuzzing, 109–122
 - launch_cvd*, 125
 - LibAFL, 6, 30–31
 - libaudioutils, 19–20

- libbinder, 110
 - API contracts, 111–112
 - IPC runtime, 111
 - parceling protocol, 111
 - security vulnerabilities, 113–115
- libFuzzer
 - vs. AFL++, 29–30
 - Android integration, 29
 - LKL-based fuzzer harnesses, 132, 134
 - with LLM-generated inputs, 180
 - output, 85
- libfuzzer_macro.h*, 35
- liblog, 19
- libpng*
 - coverage reports, 41–42
 - fuzz harness, 80–85
 - image-processing library, 73
- libprotobuf-mutator library, 35–37
- libraries, 44
 - complexity, 74
 - contributor activity, 69–70
 - critical components, 73
 - dangerous operations, 74–75
 - dependent libraries, 66–67
 - fuzzing, 75–76
 - instability, 66
 - lines changed and inserted, 70
 - past performance, 70–71
 - privilege level of process, 71
 - reachability, 66–67
 - selecting fuzz targets, 65–71
 - update frequency, 68–69
- libvirt, 18
- Linux kernel, 44
 - driver, 111
- Linux Kernel Library (LKL), 123, 132–135
 - fuzzing harness, 133–134
 - high-level workflow for LKL fuzzer, 134
 - human interface device, 133–134
 - object file (*lkl.o*), 133
 - vs. Syzkaller, 135
- Linux user ID (UID), 56
- Linux VMs, 169
- Lizard, 74
- LLAMAFUZZ, 179, 182, 184

- LLMs. *See* large language models
- llvm-cov command, 40
- LLVMFuzzerTestOneInput(), 35, 78, 81, 86, 100, 116, 134
- llvm-profdata, 39
- local attack, 103
- lockbox, 138–144, 150–151
- lockbox.c*, 138–139, 141–142
- lockbox_cleanup_module(), 142, 146
- lockbox device, 142
- lockbox.h*, 138, 154
- lockbox_init_module(), 142, 146
- lockbox_open_flags, 151
- lockbox_repro.c*, 165
- Log4Shell vulnerability, 101
- longjmp buffers, 101
- lpm_fuzzer.cpp*, 35–36
- lunch command, 21

M

- machine learning accelerators, 145
- magic_value, 37
- maintainability, 73
- mandatory access control (MAC), 45, 56
- manual parsing, 32–34
 - constant size checks, 33
 - pointer arithmetic and casting, 33
- many-shot jailbreaking, 177
- maybe-dirty suffix, 126
- mediacodec, 59
- mediaextractor, 103
- mediaserver, 59–60
- memcpy(), 34
- __memcpy_aarch64_simd(), 99
- memory leaks, fixing, 86–87
- memory-mapped I/Os (MMIOs), 174
- MemorySanitizer (MSan), 28
- Memory Tagging Extension (MTE), 11, 28–29, 47
- Microdroid, 51
- microsegmentation, 50
- Microsoft Banned Function List, 74
- miscdevice, 146
- MITRE, 70–71, 104
- mocking
 - disabling checks, 7
 - in unit testing, 12

- mode parameter, 151
- MODE_WORLD_READABLE support, 46
- module_exit(), 139
- module-info.json*, 66–67
- module-info target, 66
- module_init(), 139
- MODULE_ macros, 139
- MTE (Memory Tagging Extension), 11, 28–29, 47
- mul(), 6
- mutation-based fuzzers, 5–6
- mutator, 30
- mutex_lock(), 149
- mutex_unlock(), 149
- mutual distrust, 51
- myTestFunction(), 32–33

N

- native symbol registration, 67
- NFC chips, 145
- null pointer dereference (NPD), 5–6, 100, 102

O

- observer, 30
- OOB READ error, 102
- OOB WRITE error, 102, 113
- openat\$lockbox, 150
- open_mode, 151
- OpenSSL library, 101
- OSS-Fuzz, 2, 4, 38, 178–179
- OSS-Fuzz-Gen, 178, 184
- out-of-bounds (OOB) access
 - array, 10–11
 - read, 104, 106
 - writes, 101, 134
- out-of-memory exception, 10
- out.proffraw* file, 39
- overly aggressive de-duplication, 100

P

- Package Manager, 44
- PADLOCK_CLEAR, 149, 153
- PADLOCK_MAGIC, 145
- PADLOCK_TURN, 149, 153
- panic(), 159
- Parcel class, 111–113
 - data buffer, 114

- path traversal bugs, 11
- permissions, 53–55. *See also* SELinux
 - app requests, 55
 - escalation attacks, 55–56
 - install-time permissions, 54
 - request dialogs, 54
 - runtime permissions, 54
- physical attack, 104
- physical device, preparation, 17–18
- pKVM (protected kernel-based virtual machine), 51, 98
- platform_app, 57
- platform/manifest*, 16
- `png_create_read_struct()`, 81, 84
- `png_init_io()`, 81–82, 86
- `png_read_png()`, 81–82, 86
- pointers, 101
- POSIX environment, 169, 174
- `pr_emerg()`, 160
- `pr_err()`, 160
- `pr_info()`, 160
- `priv_app`, 57
- privilege level, 102–103
- progress evaluation, 7–10
- Project Treble, 43, 52–53
- prompt injection, 176
- proof-of-concept (PoC), 100
- protected kernel-based virtual machine (pKVM), 51, 98
- protected virtual machines (pVMs), 51
- protection-level flags, 55
- Protocol Buffers (Protobufs), 35–36
 - mutator, 36–37
- proximal attack, 103
- proximal information, 104
- `pr_warn()`, 160

Q

- quantitative analysis of reachability, 66
- quick emulator (QEMU)
 - hypervisor, 18

R

- `rand()`, 4–5
- RandomBinder, 115

- random inputs generation, 4–7
- reachability analysis, 75
- red zones, 29
- remote attack, 103
- remote procedure call (RPC), 128
- repo, 16–17
- reproducibility of crash, 96–97
- `requestPermissions()`, 55
- `rlimit`, 114
- rooting, 50
- RSA signatures, 6
- run-extract.sh*, 155
- runtime
 - feedback, 9
 - permissions, 54
 - semantics, 112

S

- sandboxing, 45, 56
- sanitizer output, 93
- sanitizers, 27–29.
 - See also* crash analysis
 - AddressSanitizer (ASan), 28, 42
 - MemorySanitizer (MSan), 28
 - Memory Tagging Extension (MTE), 28, 42
 - UndefinedBehaviorSanitizer (UBSan), 28
- SANITIZE variables, 22
- scheduler, 30
- secure elements (SEs), 43, 47, 50, 102
- secure monitor, 48
- secure monitor call (SMC), 49
- security context, 56
- seed corpus, 90–91
- SELinux, 43, 45–46
 - `audit2allow`, 59
 - `chcon`, 59
 - domains, 56–57
 - how it works, 56–58
 - kernel enforcement, 57–58
 - policy, 57–59
 - restorecon, 59
 - shortcomings, 59
- `setjmp()`, 81, 84
- `set_value()`, 96

- severity assessment, 100
 - attack vector, 103–104
 - behavior, 101
 - code execution, 101
 - denial of service (DoS), 102
 - information disclosure, 101–102
 - final determination, 104–106
 - privilege level, 102–103
- shadow memory, 28
- shared_libs tag, 19
- Shellshock bug, 4
- shuffle inconsistency (SI) attack, 177
- silent memory corruptions, 27
- social engineering, 55
- Soong build system, 15, 19
 - build targets, 19–21
 - building-time options, 22
- sprintf(), 74
- SQLite database, 44
- Stable AIDL, 53
- stack overflows, 101
- Stagefright crisis, 4, 59
- stateful program, 7
- str*(), 74
- StrongBox, 50
- structure-aware fuzzing, 5, 120–121
- structured communication, 75
- Structured Query Language (SQL), 11
- swallow_error(), 84
- swallow_warning(), 84
- sync subcommand, 79
- syscalls, 124, 128, 131–132, 135, 150
- system_app, 57
- system_data_file, 56
- system-level protections, 43
- system services, 109
- syz-env, 127
- syz-executor, 128
- Syzkaller, 30, 123–124, 126, 137. *See also* kernel fuzzing; Linux Kernel Library (LKL)
 - configuration options, 128–131
 - fields, 129
 - serial console, 130
 - descriptions
 - describing lockbox, 150–151
 - describing padlock, 151–153
 - extracting constants, 153–156
 - fuzzing, 156–157
 - lockbox driver, 140–144
 - new descriptions, 157–159
 - padlock driver, 144–149
 - reproducer, running, 163–165
 - GitHub, 127
 - vs. LKL, 135
 - overview, 128
 - sandboxed environment, fuzzing
 - inside, 131–132
 - setting up, 127–128
 - Syscalls page, 156–157
 - syz-manager, 127–128
 - web UI, 131
- Syzlang, 30, 149–153
 - extracting constants, 153–156
 - lockbox descriptions, 150–151
 - padlock descriptions, 151–153
- syz-manager, 128

T

- TEEs. *See* trusted execution environments
- test case, 93
- test-driven development, 12
- time to remediation (TTR), 76
- true random number generator (TRNG), 50
- trusted applications, 48
- trusted execution environments (TEEs), 43, 47–50, 103
 - Keystore service, 49
- TrustZone, 48

U

- UndefinedBehaviorSanitizer (UBSan), 11, 28, 126
- unit testing, 12
- Unix-like systems, 123
- Unix pipeline, 70
- untrusted_app, 56–57
- untrusted apps, 103
- use-after-free (UAF), 101

V

- valueprofile, 6
- Vendor Test Suite, 53

- `virtual_device_x86_64_external_modules`, 140
- virtualization, 98
- virtual machines (VMs), 18
 - running in QEMU, 128
- vulnerabilities in libraries, 66
- vulnerability management team, 100

W

- web user interface, 21
- white-box fuzzing, 179–181. *See also*
 - black-box fuzzing; gray-box fuzzing

- Window Manager, 44
 - `-Wno-unused-parameter`, 79
- work profiles, 45
- world-readable files, 46

X

- `x86_64/lockbox` rule, 140
- XML (Extensible Markup Language), 5

Z

- zero-length input, 5